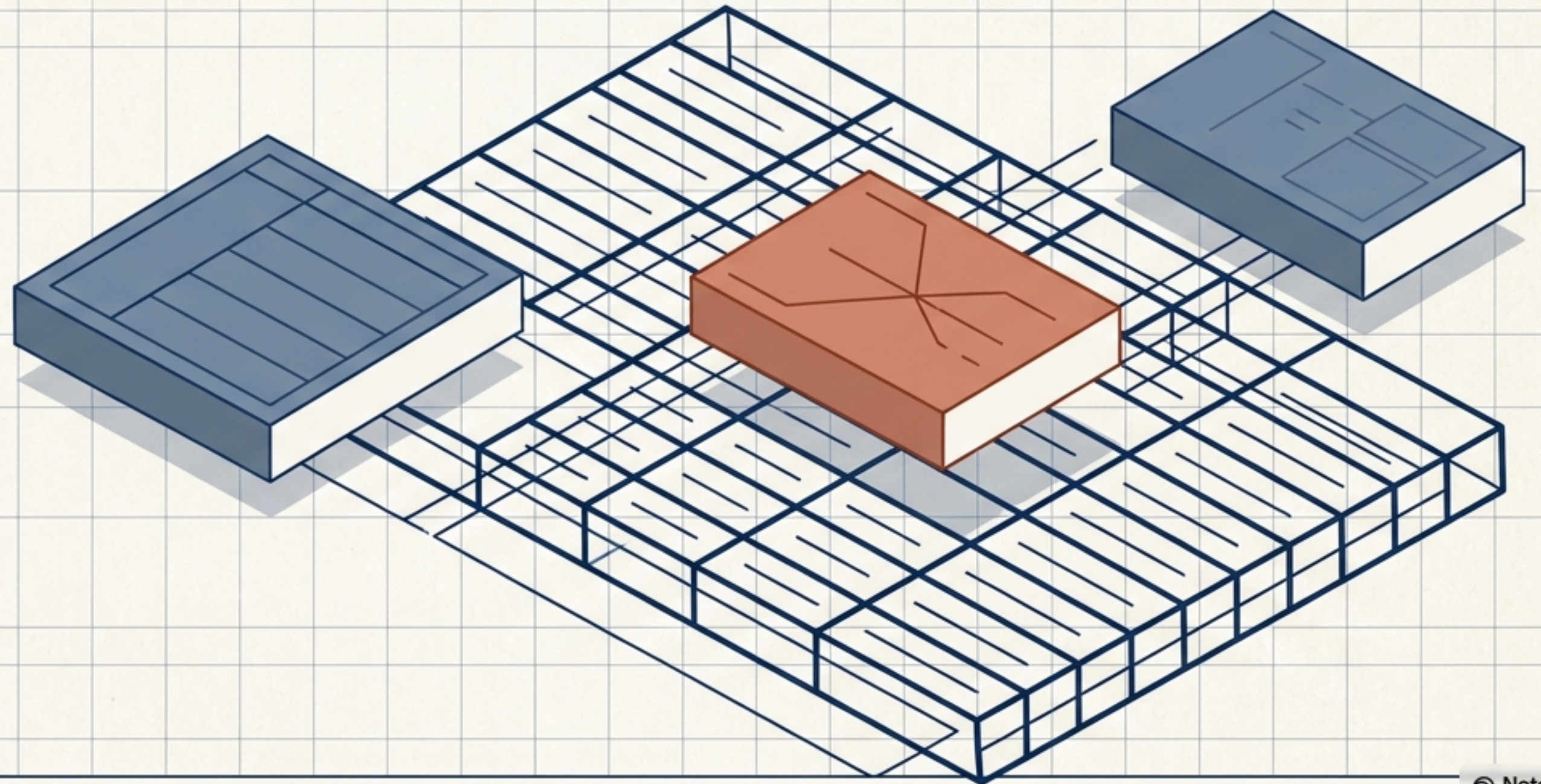


Data Architecture with Pandas

A Visual Blueprint for Renaming
and Combining Datasets



THE CONSOLIDATION PROBLEM

Raw data rarely arrives ready for analysis. It comes from disparate sources with overlapping schemas, misnamed headers, and fragmented structures.

Table A: Sales_Data_Raw

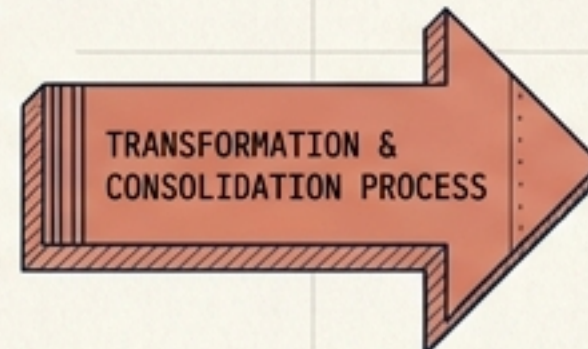
CustomerID	TransactionID	Date	Product
CUS121	1801	9/123/21	Product
CUS112	1892	9/123/21	Region
C05133	1023	9/135/21	Product
C05154	1084	9/123/21	Region
C05185	1805	9/153/21	Product
CUS136	1876	9/133/21	
C05178	1878	9/125/23	
C09183	1079	9/133/21	
...	1030	9/33/21	Region
...	...	...	...

PROBLEM 1: Inconsistent Naming
e.g., 'cust_id' vs 'customer_ID'

Table B: CRM_Info_Fragment

Table/S	...	...	...	...
...	...	87008L 3	...	...
...	...	...	...	...
CRBS3	...	...	...	...
CRS21	...	...	...	...
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...

PROBLEM 2: Disconnected Sources
No shared key, data silos



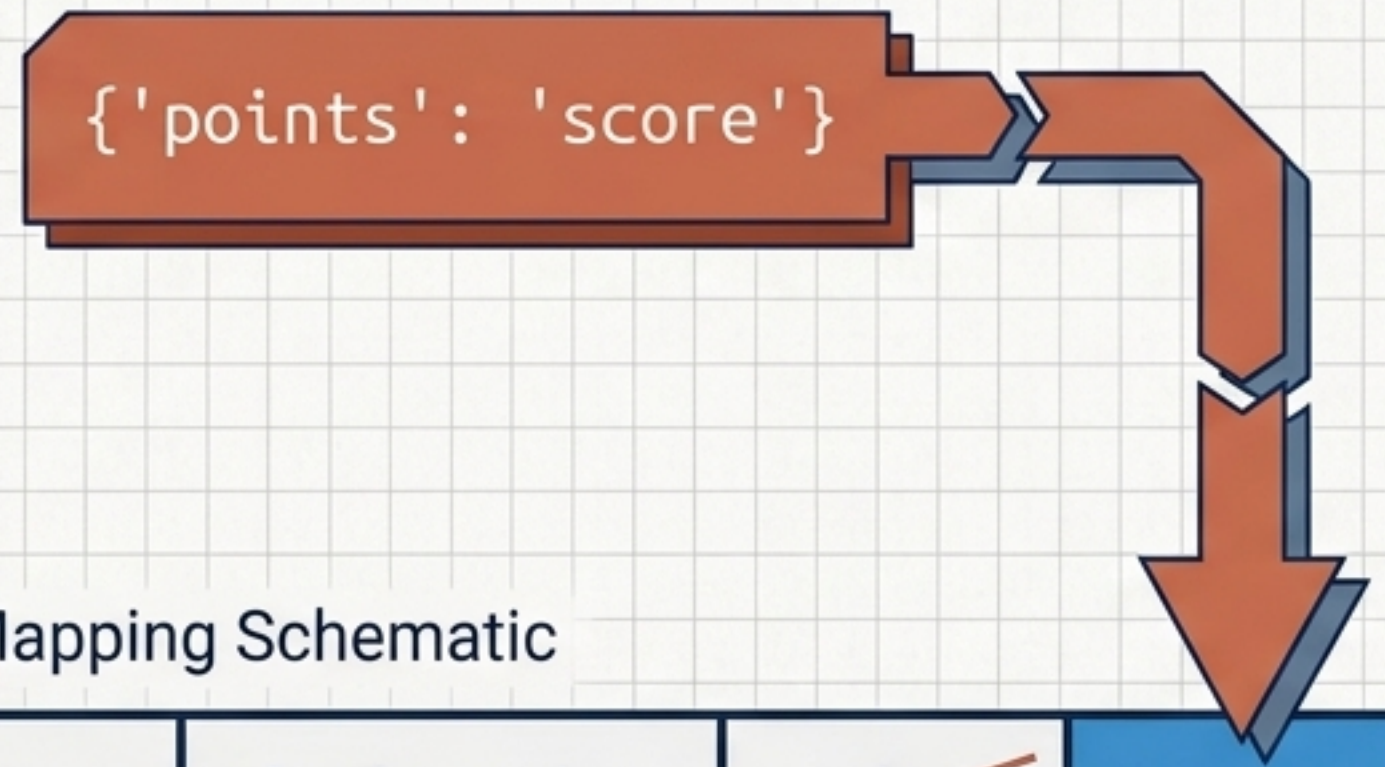
ARCHITECTED DATA TABLE: Unified_Sales_CRM_Model

CustomerID	TransactionID	Date	Product	Region	SalesAmount	LeadSource	Status
CUS135	22588	9/21/2023	Product	USA	\$25.80	Creator	Complete
CUS126	22581	9/23/2022	Rogon	South	\$25.80	LeadSource	Status
CUS127	25502	9/21/2022	Product	USA	\$15.80	Noneay	Complete
CUS128	22503	9/21/2023	Regeon	South	\$15.80	LeadSource	Status
C05183	25584	9/25/2023	Product	USA	\$25.80	Creator	Complete
C05128	25585	9/23/2023	Rogon	South	\$25.80	LeadSource	Status
C05188	23606	9/15/2023	Product	USA	\$15.80	Creator	Complete
C05181	25887	9/18/2022	Rogon	South	\$15.80	LeadSource	Status
C05122	28988	9/19/2023	Product	USA	\$25.80	Noneey	Complete
C09533	23989	9/15/2023	Japan	South	\$15.80	LeadSource	Status
C09522	25989	9/15/2023	Product	USA	\$15.80	LeadSource	Status

Column Micro-Adjustments: rename()

The `rename()` function resolves offending column names by accepting a dictionary mapping old values to new ones.

```
reviews.rename(columns={'points': 'score'})
```



Dictionary Mapping Schematic

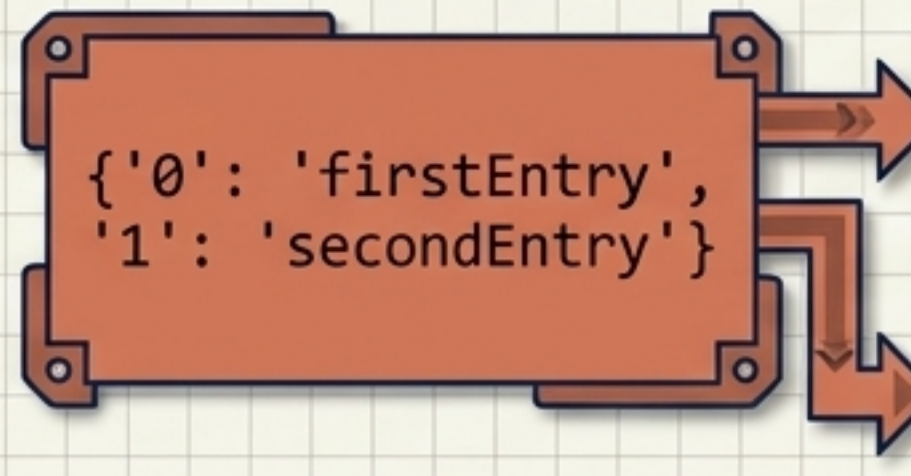
	country	designation	points	score
1	Italy	Vulkà Bianco	87	
2	Portugal	Avidagos	87	

Index Micro-Adjustments

`rename()` can also target index values using the `index` parameter, though renaming individual index values is rare.

```
reviews.rename(index={0: 'firstEntry', 1: 'secondEntry'})
```

For comprehensive index replacement, `set_index()` is usually more convenient than mapping individual values via `rename()`.



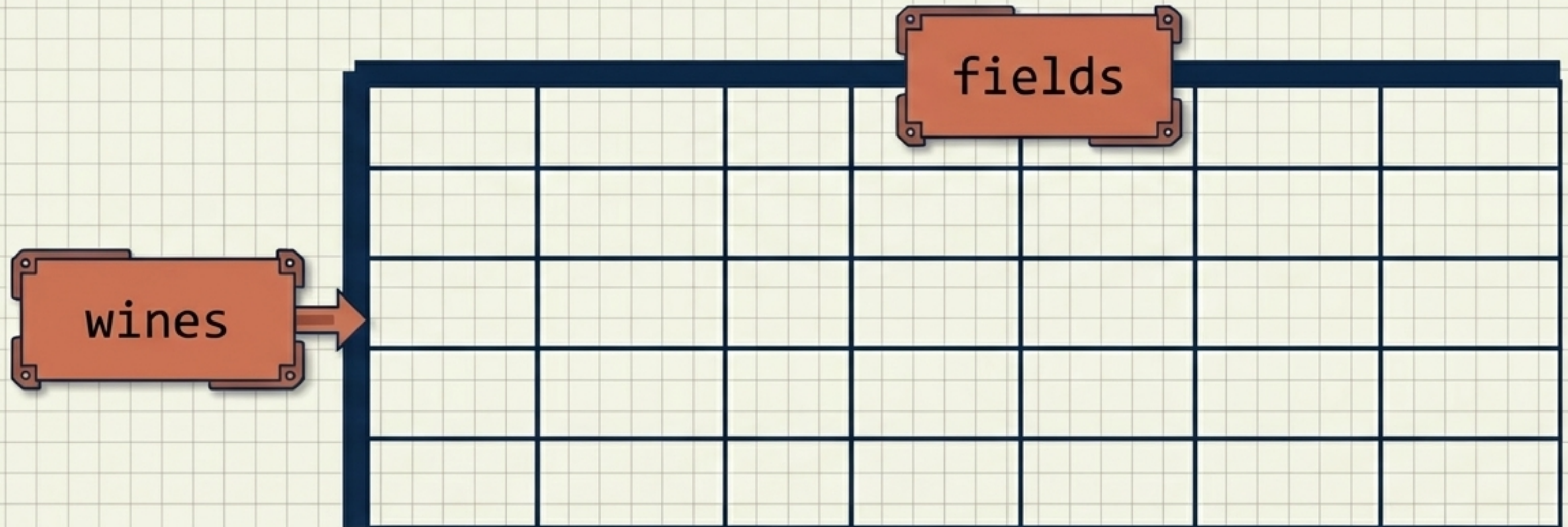
The diagram shows a mapping box with the dictionary `{'0': 'firstEntry', '1': 'secondEntry'}`. Two arrows point from this box to the index column of a table. The first arrow points from the original index '0' to the new label 'firstEntry'. The second arrow points from the original index '1' to the new label 'secondEntry'. The original index values '0' and '1' in the table are crossed out with red lines.

		country	score
0	firstEntry	Italy	87
1	secondEntry	Portugal	87

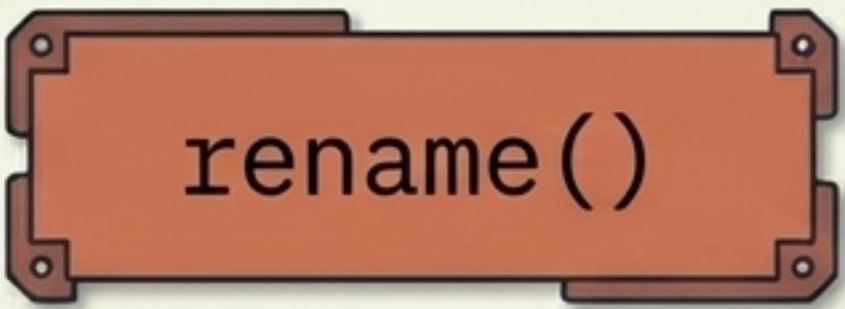
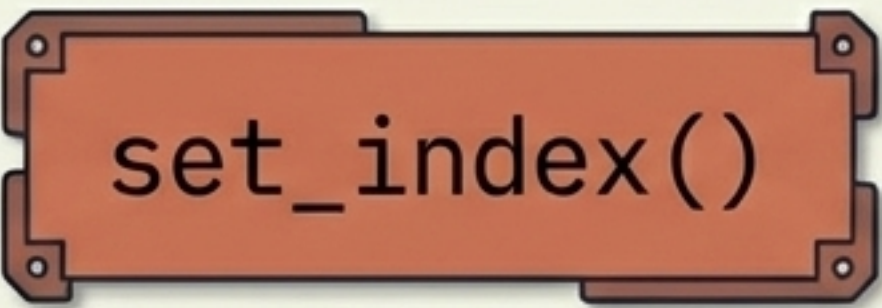
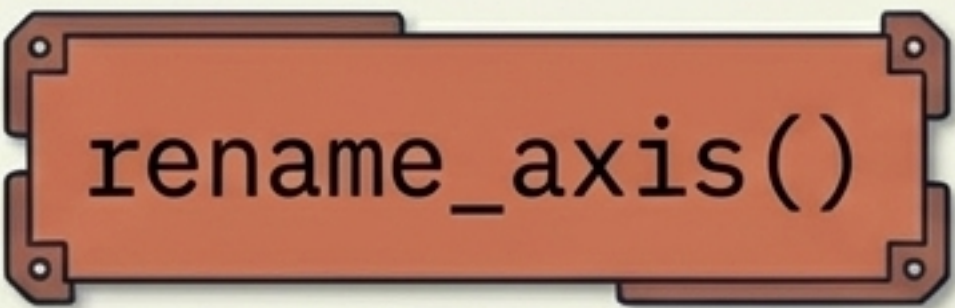
Labelling the Blueprint: rename_axis()

Both the row index and the column index possess their own name attributes. `rename_axis()` assigns a master label to the entire axis.

```
reviews.rename_axis('wines', axis='rows').rename_axis('fields', axis='columns')
```

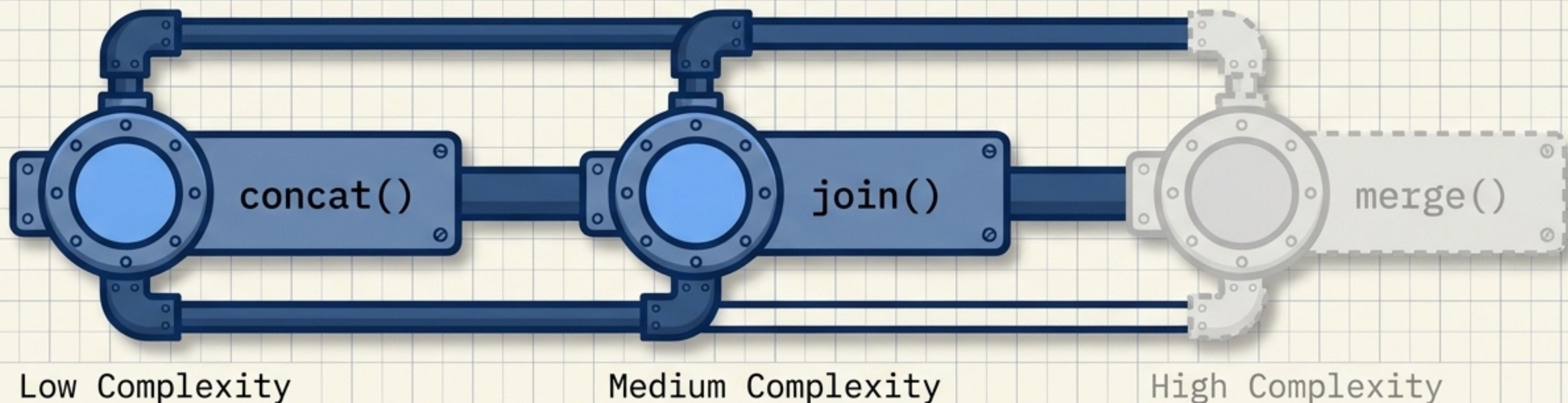


The Renaming Diagnostic Matrix

Function	Target	Primary Use
 <code>rename()</code>	Specific Elements	Changing individual column headers or row names via dictionary mapping.
 <code>set_index()</code>	Entire Index	Swapping out an entire existing index for a new one.
 <code>rename_axis()</code>	Axis Attribute	Assigning a master title to the entire row or column structure.

Macro-Adjustments: Combining Data

Pandas offers three core methods for non-trivial combinations, scaling in complexity. We focus on the two most foundational: `concat()` and `join()`, as `join()` simplifies most of what `merge()` can achieve.

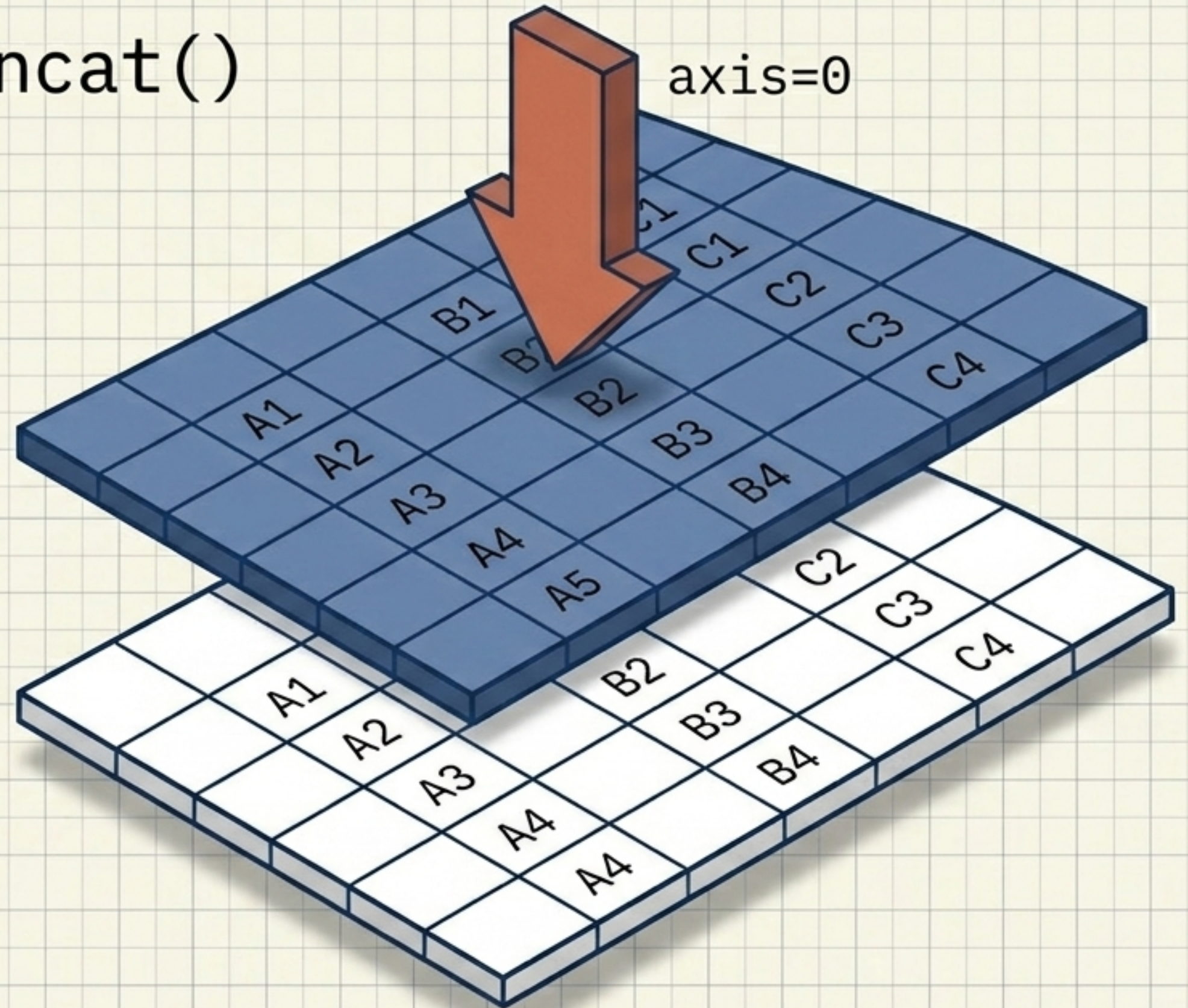


The Stacking Tool: `concat()`

Given a list of elements, `concat()` simply smushes them together along a defined axis.

It is the ideal tool when you have fragmented data with identical fields.

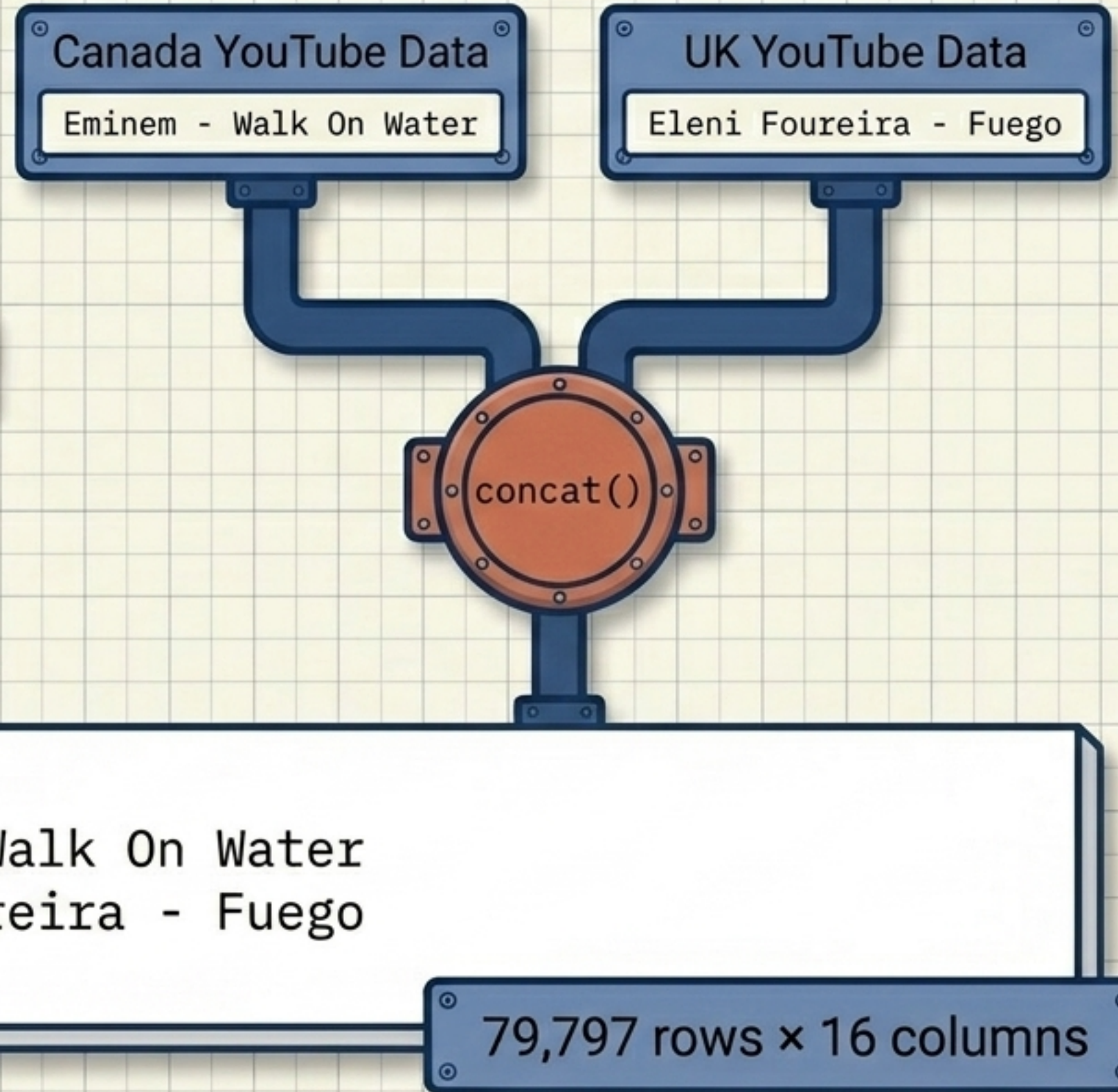
Mechanic Note: Assumes column schemas are identical.



concat() in Application

To study multiple countries simultaneously, stack their independent files.

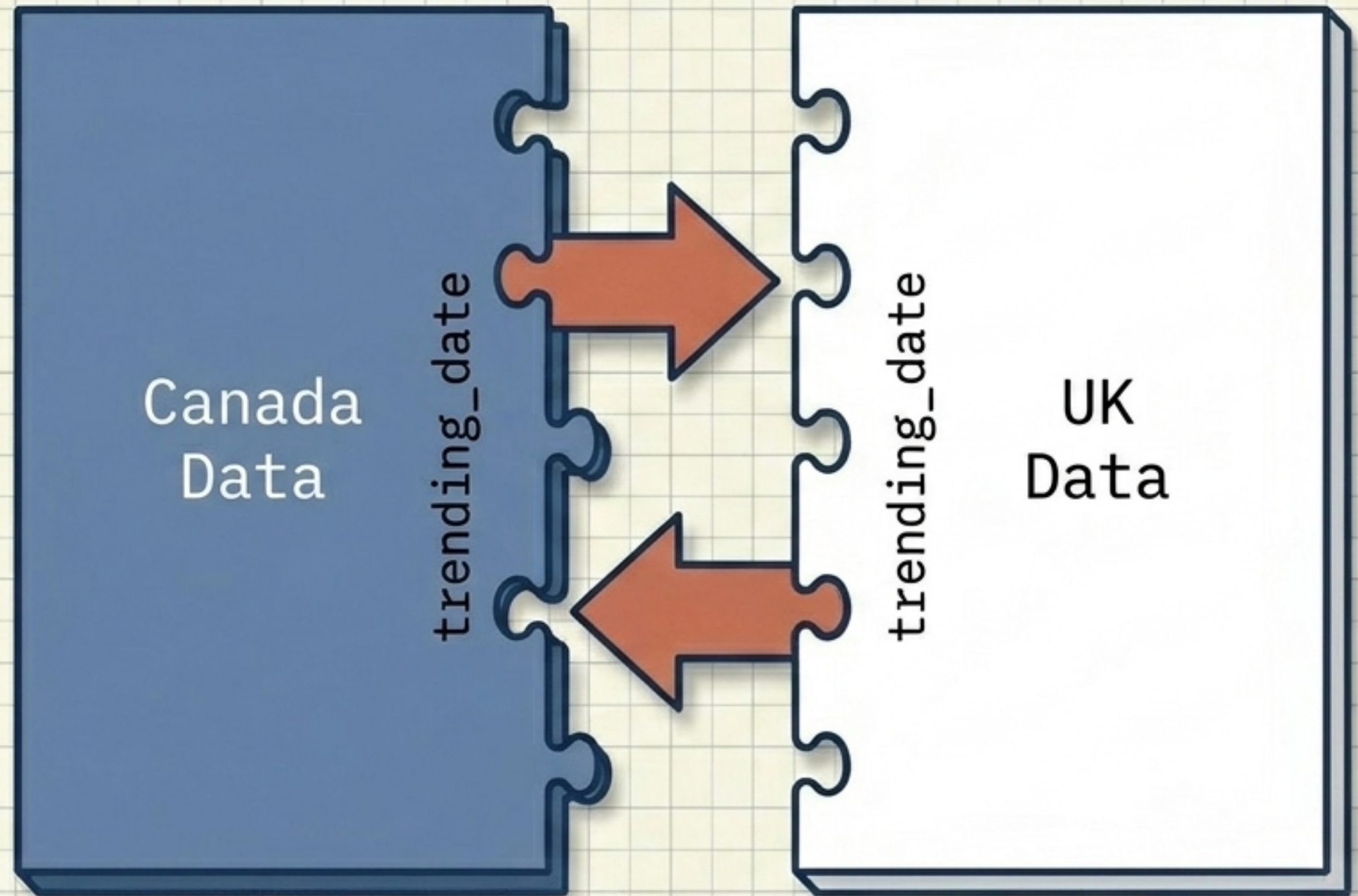
```
pd.concat([canadian_youtube, uk_youtube])
```



The Interlocking Tool: `join()`

Combining datasets horizontally.
`join()` links different
DataFrames together based
on a common index.

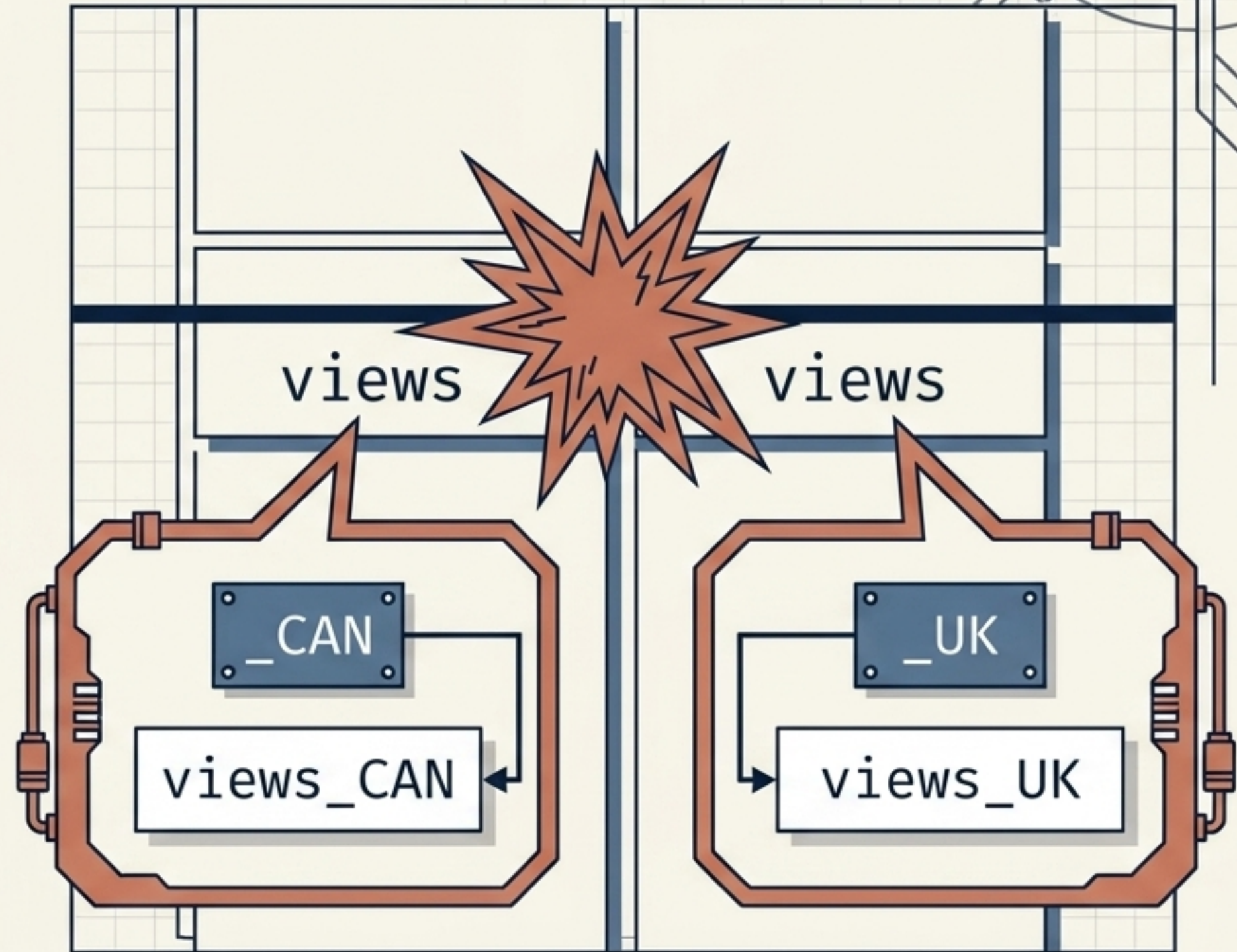
Use Case: Pulling down videos
that were trending on the exact
same day in both Canada and
the UK.



Resolving Overlaps in join()

Because both datasets share identical column names, forcing them together creates a naming collision. The `lsuffix` and `rsuffix` parameters automatically append identifiers to resolve this.

```
left.join(right, lsuffix='_CAN',  
          rsuffix='_UK')
```



The Combining Diagnostic Matrix

Function	Mechanic	Primary Use Case
<code>concat()</code>	Vertical/Horizontal Stacking	Smushing structurally identical DataFrames together.
<code>join()</code>	Index Alignment	Merging different structural data side-by-side using a shared index.
<code>merge()</code>	Column/Index Alignment	Complex relational database-style joining (largely covered by <code>join()</code>).

The Data Architect's Decision Tree

What does your dataset need?

I need to fix
structure/labels.

Changing specific
headers?

Use `rename()`

Changing the
master axis name?

Use
`rename_axis()`

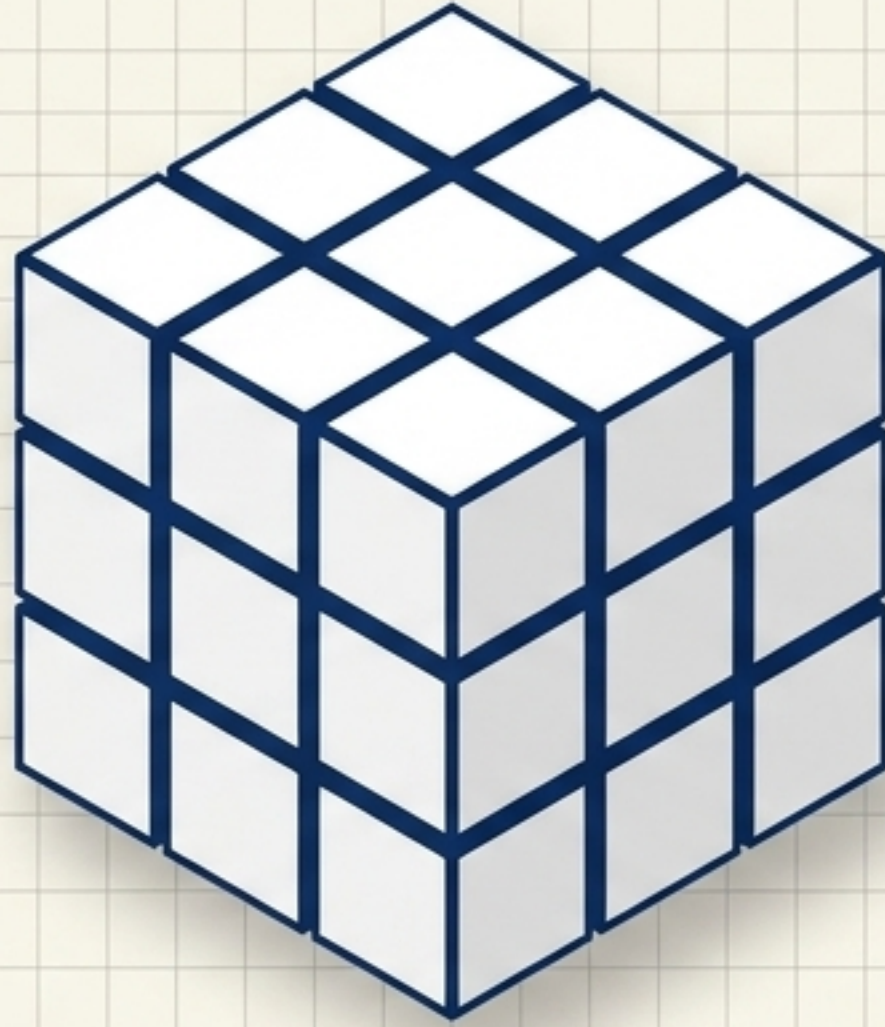
I need to add
more data.

Are the datasets
identically
structured?

Use `concat()`

Do they share a
common index but
differ elsewhere?

Use `join()`



Data, Architected.

Keep this blueprint close. Mastering these fundamental structural tools transforms messy inputs into analytical clarity.